

`symmetric_difference`

- 두 집합의 합집합에서 교집합을 뺀 집합이다.

- 사용방법

```
'set1' ^ 'set2' # set1과 set2의 대칭차집합 연산자  
'set1'.symmetric_difference('set2') #set1과 set2의 대칭차집합 함수
```

- 예제

```
setA={1,2,3,4}  
setB={3,4,5,6}  
  
print(setA^setB)  
  
print(setA.symmetric_difference(setB))  
  
{1, 2, 5, 6}  
{1, 2, 5, 6}
```

`symmetric_difference_update`

- 서로 다른 세트 데이터를 결합했지만, 중복되지 않은 데이터만 출력하고 싶을 때 사용.

- 세트 데이터가 서로 결합은 되지만 중복된 값은 배제한다.

- 사용방법

```
A.symmetric_difference_update(B)
```

- 예제

```
a= {1,2,3,4}  
b= {3,4,5,6}  
  
a.symmetric_difference_update(b)  
print(a)  
print(b)  
  
{1, 2, 5, 6}  
{3, 4, 5, 6}
```

Union(합집합)

- 합집합은 두 집합의 모든 원소를 포함하는 집합이다.
- 기존에는 ' | ' 연산자를 사용했지만, union() 함수를 사용할 수 있다.
- 사용 방법

```
setA | set B    #setA와 setB의 합집합 연산자  
setA.union(setB) #setA와 setB에 대한 합집합 함수
```

- 예제

```
setA={1,2,3,4}  
setB={3,4,5,6}  
  
print(setA | setB)  
print(setA.union(setB))  
  
{1, 2, 3, 4, 5, 6}  
{1, 2, 3, 4, 5, 6}
```

update(업데이트)

- 집합에 다른 집합이나 iterable의 원소들을 추가하는 기능을 한다.
- 여러 원소를 한꺼번에 추가할 때 사용한다.

- 사용 방법

```
set.update(iterable1, iterable2, ...)
```

- 예제.

```
a = {1,2,3,4}  
b = {3,4,5,6}  
  
a.update(b)  
print(a)  
  
{1, 2, 3, 4, 5, 6}
```